

Documentação essencial do projeto UniSalas

1. Sobre o projeto

O UniSalas é um sistema web desenvolvido em Laravel para reserva e gerenciamento de salas acadêmicas. A aplicação centraliza o processo de solicitação, aprovação e acompanhamento de reservas, reduzindo o uso de controles manuais e melhorando a comunicação entre professores e administração.

O sistema possui dois perfis principais:

- **Administrador:** gerencia usuários, blocos, salas, manutenções, reservas, avisos e mensagens.
- **Professor:** solicita reservas, acompanha respostas, consulta histórico e troca mensagens com a administração.

2. Tecnologias utilizadas

Tecnologia	Uso no projeto
PHP 8.2+	Linguagem principal
Laravel 12	Framework back-end
MySQL/MariaDB	Banco de dados
Blade	Templates das telas
Bootstrap 5	Layout e componentes visuais
Font Awesome	Ícones
SweetAlert2	Alertas e feedback visual
Composer	Dependências PHP
Node.js/NPM/Vite	Build e desenvolvimento front-end
PHPUnit	Testes automatizados

3. Requisitos para executar

- PHP 8.2 ou superior.
- Composer.
- Node.js e NPM.
- MySQL ou MariaDB.
- XAMPP, Laragon ou ambiente equivalente.

4. Instalação

Clone ou copie o projeto para uma pasta local. No XAMPP, recomenda-se:

```
C:\xampp\htdocs\reserva-salas
```

Instale as dependências:

```
composer install
npm install
```

Crie o arquivo `.env`:

```
copy .env.example .env
```

Gere a chave da aplicacao:

```
php artisan key:generate
```

Configure o banco no .env:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=reserva_salas
DB_USERNAME=root
DB_PASSWORD=
```

Crie o banco reserva_salas no phpMyAdmin e execute:

```
php artisan migrate --seed
```

Tambem existe um script SQL pronto em:

```
docs/database_unisalas.sql
```

5. Como rodar o sistema

Em um terminal:

```
php artisan serve
```

Em outro terminal:

```
npm run dev
```

Acesse:

```
http://127.0.0.1:8000
```

6. Usuarios de teste

Administrador:

```
E-mail: l@gmail.com
Senha: 12345678
```

Professor:

```
E-mail: professor@unisalas.local
Senha: 12345678
```

7. Funcionalidades principais

Administrador

- Login administrativo.
- Dashboard com indicadores.
- Cadastro e gerenciamento de usuarios.
- Cadastro e gerenciamento de blocos.
- Cadastro e gerenciamento de salas.
- Controle de manutencao de blocos e salas.

- Aprovacao, recusa e cancelamento de reservas.
- Aprovacao ou recusa de reservas recorrentes em lote.
- Historico geral de reservas com filtros.
- Publicacao de avisos.
- Mensagens diretas com professores.

Professor

- Login como professor.
- Painei com resumo das reservas.
- Solicitacao de reserva simples.
- Solicitacao de reserva recorrente.
- Verificacao de disponibilidade de sala.
- Consulta de minhas reservas.
- Consulta de historico.
- Desistencia de reserva.
- Visualizacao de avisos.
- Mensagens diretas com administradores.

8. Estrutura principal do projeto

```

app/Http/Controllers/AuthController.php
app/Http/Controllers/AdminController.php
app/Http/Middleware/CheckAdmin.php
app/Models/User.php
app/Models/Bloco.php
app/Models/Sala.php
app/Models/Reserva.php
app/Models/Aviso.php
app/Models/MensagemDireta.php
app/Models/NotificacaoVisualizada.php
routes/web.php
resources/views/
database/migrations/
database/seeders/

```

9. Banco de dados

As principais tabelas do sistema sao:

Tabela	Finalidade
users	Armazena administradores e professores
blocos	Armazena os blocos da instituicao
salas	Armazena as salas vinculadas aos blocos
reservas	Armazena solicitacoes e reservas
avisos	Armazena comunicados da administracao
mensagens_diretas	Armazena mensagens entre usuarios
notificacao_visualizadas	Controla notificacoes ja visualizadas

Relacionamentos principais:

- Um bloco possui varias salas.

- Uma sala pertence a um bloco.
- Um professor pode possuir varias reservas.
- Uma reserva pertence a um professor e a uma sala.
- Uma mensagem possui remetente e destinatario.

10. Regras de negocio principais

- Apenas usuarios autenticados acessam o sistema.
- Rotas administrativas exigem perfil de administrador.
- Reservas sao verificadas por sala, data e periodo.
- Uma sala nao pode ser reservada no mesmo periodo quando ja existe conflito.
- Reservas recorrentes sao geradas semanalmente por ate tres meses.
- O administrador pode aprovar ou recusar reservas individualmente ou em lote.
- Blocos e salas em manutencao bloqueiam ou cancelam reservas afetadas.
- Professores visualizam respostas administrativas nas proprias reservas.
- Mensagens nao lidas geram indicador visual no sistema.

11. Rotas essenciais

Rota	Funcao
/login	Tela de login
/logout	Sair do sistema
/admin/dashboard	Dashboard do administrador
/admin/usuarios	Gerenciamento de usuarios
/admin/blocos	Gerenciamento de blocos e salas
/admin/reservas	Aprovacao de reservas
/admin/historico-completo	Historico administrativo
/professor/painel	Painel do professor
/professor/solicitar	Solicitar reserva
/professor/minhas-reservas	Minhas reservas
/professor/historico	Historico do professor
/mensagens	Lista de mensagens
/chat/{id}	Conversa direta

12. Comandos uteis

Limpar cache:

```
php artisan optimize:clear
```

Rodar testes:

```
php artisan test
```

Recriar banco com dados iniciais:

```
php artisan migrate:fresh --seed
```

Gerar build do front-end:

```
npm run build
```

13. Arquivos de apoio

- Script SQL completo: docs/database_unisalas.sql
- Documentacao completa: docs/DOCUMENTACAO_COMPLETA_UNISALAS.md
- Documentacao essencial: docs/DOCUMENTACAO_ESSENCIAL_UNISALAS.md

14. Melhorias futuras

- Criar relatorios gerenciais.
- Melhorar separacao do AdminController em controllers menores.
- Ampliar testes automatizados.
- Criar exportacao de historico.
- Integrar com sistemas academicos externos.